INFORMATION RETRIEVAL SYSTEM FOR HOSPITAL DOMAIN DOCUMENTS

Group G31

Student Name	BITS ID	Contribution
KARTHICK BABU AV	2024dc04284	100%
HARIKRISHNA M	2024dc04289	100%
KIRAN P VISWAM	2024dc04290	100%

URLs:

Git repo: https://github.com/2024dc04284-pixel/IR_G31_Assignment

Live app: https://irg31assignment-9ewwyf32rzqrvitrm3mziy.streamlit.app/

Summary

This project implements an interactive Information Retrieval System using Streamlit, NLTK and a collection of hospital-domain documents. The system demonstrates fundamental IR concepts including text preprocessing, inverted indexing, phrase query processing, dictionary search structures, and tolerant retrieval mechanisms. Various indexing and retrieval techniques such as Biword Indexing, Positional Indexing, Binary Search Trees (BST), B-Trees, K-gram Indexing, Edit Distance, Wildcard Matching, Spell Correction, and Soundex-based phonetic matching are implemented and compared. The system provides an interactive interface that allows users to experiment with different retrieval techniques and observe their effectiveness.

---

1. Introduction

Information Retrieval focuses on obtaining relevant information from large collections of documents based on user queries. Traditional search systems rely heavily on efficient indexing structures and query processing techniques to retrieve information accurately and quickly.

The objective of this project is to develop a complete Information Retrieval system capable of:

- Processing textual hospital-domain documents.
- Building searchable indexes.
- Supporting phrase queries.
- Comparing dictionary structures.
- Handling spelling errors and incomplete queries.
- Demonstrating core Information Retrieval concepts through visualization and experimentation.

The project is implemented using Python, Streamlit, NLTK, and Pandas.

---

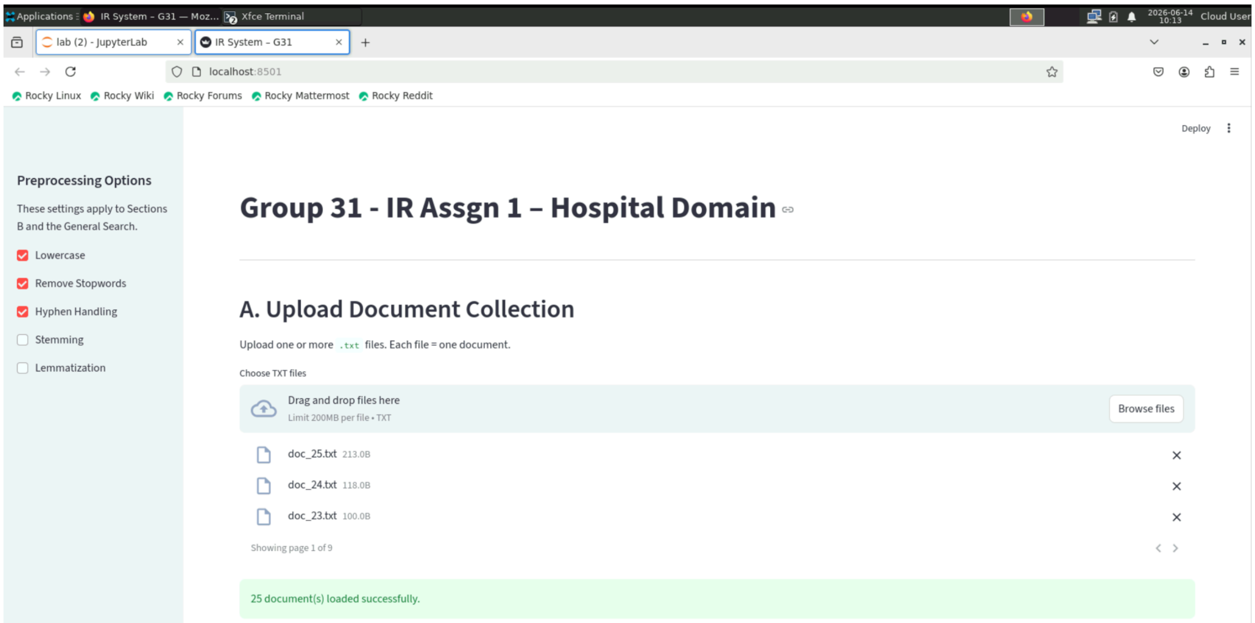
2. Dataset Description

The dataset consists of twenty-five hospital-related text documents covering various medical specialties and healthcare activities.

Examples include:

- Coronary artery bypass surgery
- Blood glucose monitoring
- Intensive care unit operations
- Oncology treatments
- Radiology imaging
- Neurology and stroke management
- Pharmacy and medication dispensing

Each document contains one or more healthcare-related concepts and serves as a document in the retrieval collection.



---

3. System Architecture

The system is divided into five major modules:

1. Document Upload Module
2. Text Preprocessing Module
3. Inverted Index Construction Module
4. Phrase Query Processing Module
5. Tolerant Retrieval Module

The workflow is:

Documents → Preprocessing → Index Construction → Query Processing → Retrieval Results

---

4. Text Preprocessing

Text preprocessing improves retrieval effectiveness by normalizing document content before indexing.

The implemented preprocessing operations include:

4.1 Lowercasing

All text is converted to lowercase.

Example:

Patient → patient

Benefits:

- Eliminates case sensitivity.
- Reduces vocabulary size.

4.2 Hyphen Handling

Hyphenated words are split into separate tokens.

Example:

post-operative → post operative

Benefits:

- Improves matching accuracy.

4.3 Tokenization

Text is divided into individual terms.

Example:

"The patient received treatment"

becomes

["the", "patient", "received", "treatment"]

4.4 Stopword Removal

Common words with little retrieval value are removed.

Examples:

- the
- is
- of
- and

Benefits:

- Reduces noise.
- Improves retrieval precision.

4.5 Stemming

Words are reduced to approximate root forms.

Examples:

- prescribing → prescrib
- patients → patient

Advantages:

- Reduces vocabulary size.

Disadvantages:

- May generate non-dictionary words.

4.6 Lemmatization

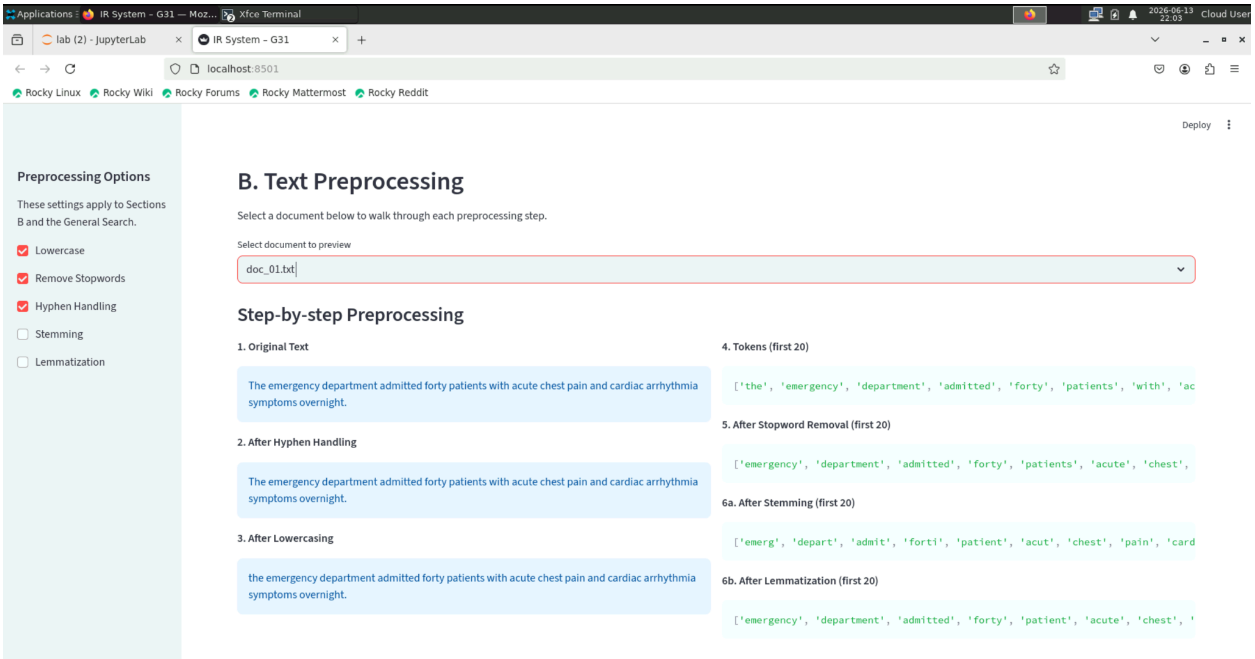
Words are reduced to valid dictionary base forms.

Examples:

- prescribing → prescribe
- injuries → injury

Advantages:

- Preserves semantic meaning.
- Better suited for medical terminology.



5. Inverted Index Construction

An inverted index maps terms to documents containing those terms.

Example:

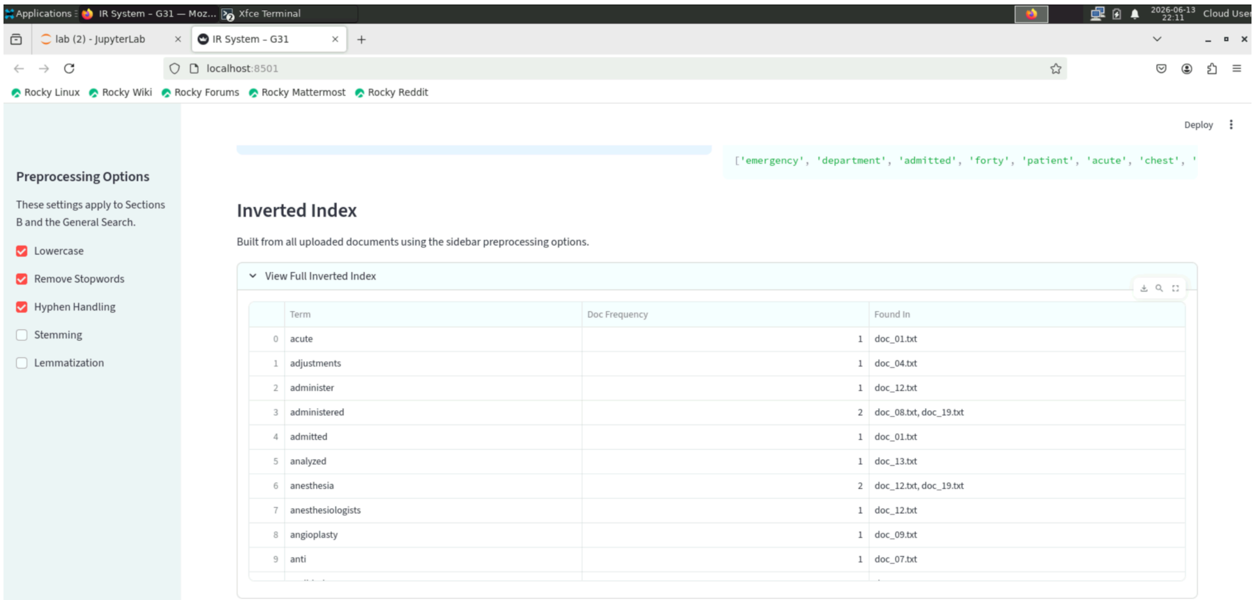
Term	Documents
patient	doc_06, doc_11
surgery	doc_06, doc_17
blood	doc_08, doc_13

Benefits:

- Fast term lookup.
- Efficient query processing.
- Foundation of modern search engines.

Complexity:

- Index construction: O(N)
- Query lookup: O(1) average



---

6. General Search

The General Search module allows users to search across all uploaded documents.

Process:

1. Query preprocessing
2. Token matching
3. Similarity scoring
4. Result ranking

Current scoring method:

Score = Matched Terms / Total Query Terms

Example:

Query:

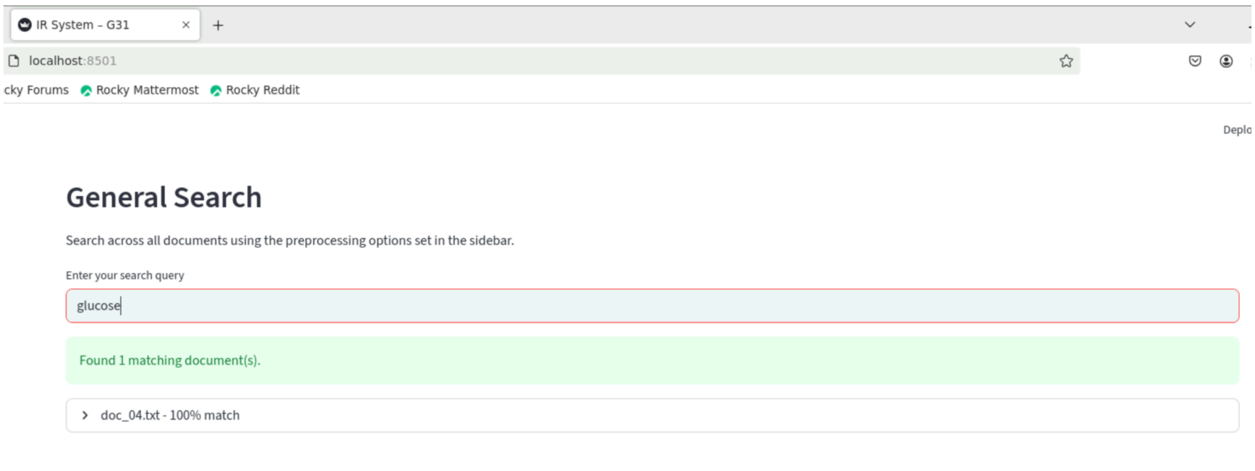
blood glucose monitoring

Matching document:

Patients with type 2 diabetes require regular blood glucose monitoring.

Result Score:

100%



---

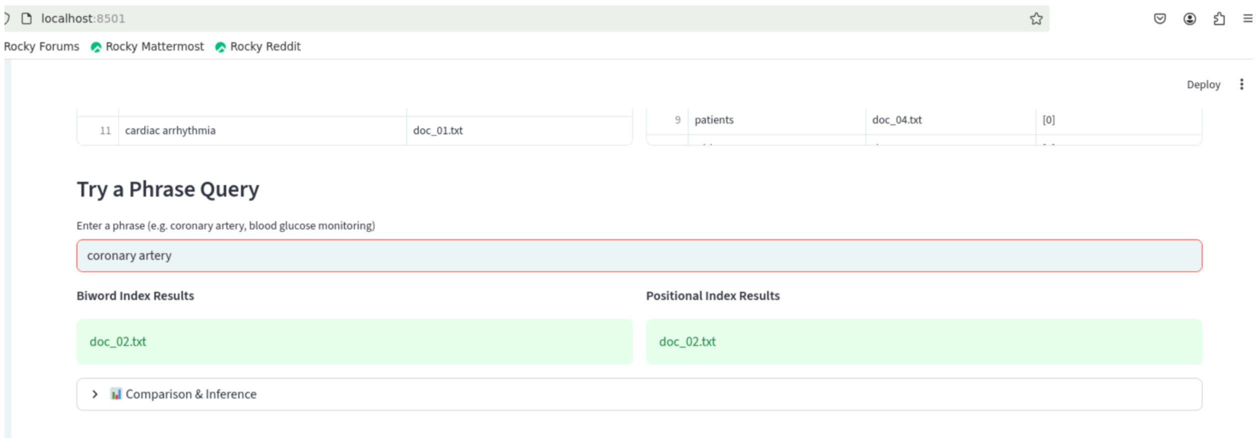
7. Phrase Query Processing

Phrase queries require exact word ordering.

Example:

"coronary artery"

Two indexing approaches are implemented.



7.1 Biword Index

Stores consecutive word pairs.

Example:

coronary artery bypass surgery

Produces:

- coronary artery
- artery bypass
- bypass surgery

Advantages:

- Compact storage.
- Fast for two-word phrases.

Limitations:

- May generate false positives for longer phrases.

7.2 Positional Index

Stores term positions within documents.

Example:

coronary → [0]
artery → [1]
bypass → [2]

Advantages:

- Supports exact phrase matching.
- No false positives.

Disadvantages:

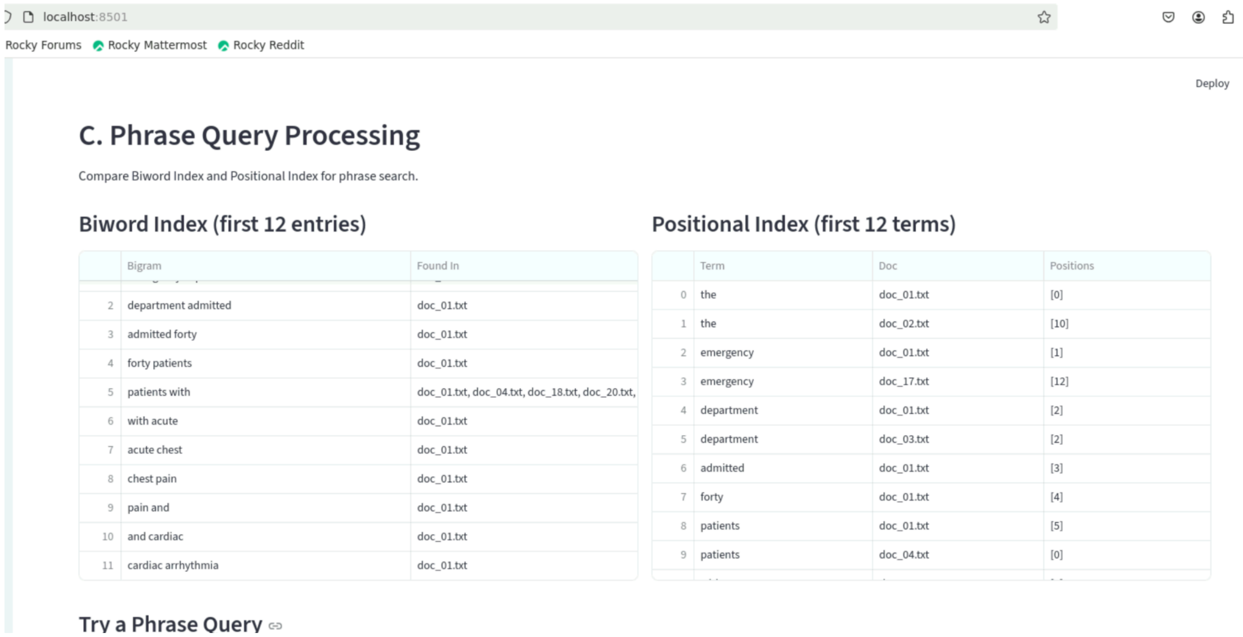
- Larger storage requirements.

Comparison

Feature	Biword Index	Positional Index
Storage	Small	Large
Accuracy	Moderate	High
Phrase Length	Limited	Any Length
False Positives	Possible	None

Conclusion:

Positional Index provides superior phrase retrieval accuracy.



---

8. Dictionary Search Structures

The dictionary consists of all unique terms appearing in the collection.

Two search structures are implemented.

8.1 Binary Search Tree (BST)

BST stores terms in hierarchical order.

Characteristics:

- Average Search: O(log n)
- Worst Case: O(n)

Problem:

Sorted insertion causes tree skewness.

8.2 B-Tree

B-Tree stores multiple keys per node and remains balanced.

Characteristics:

- Search: O(log n)
- Balanced by design
- Efficient for large collections

Advantages:

- Lower tree height
- Better scalability
- Cache-friendly structure

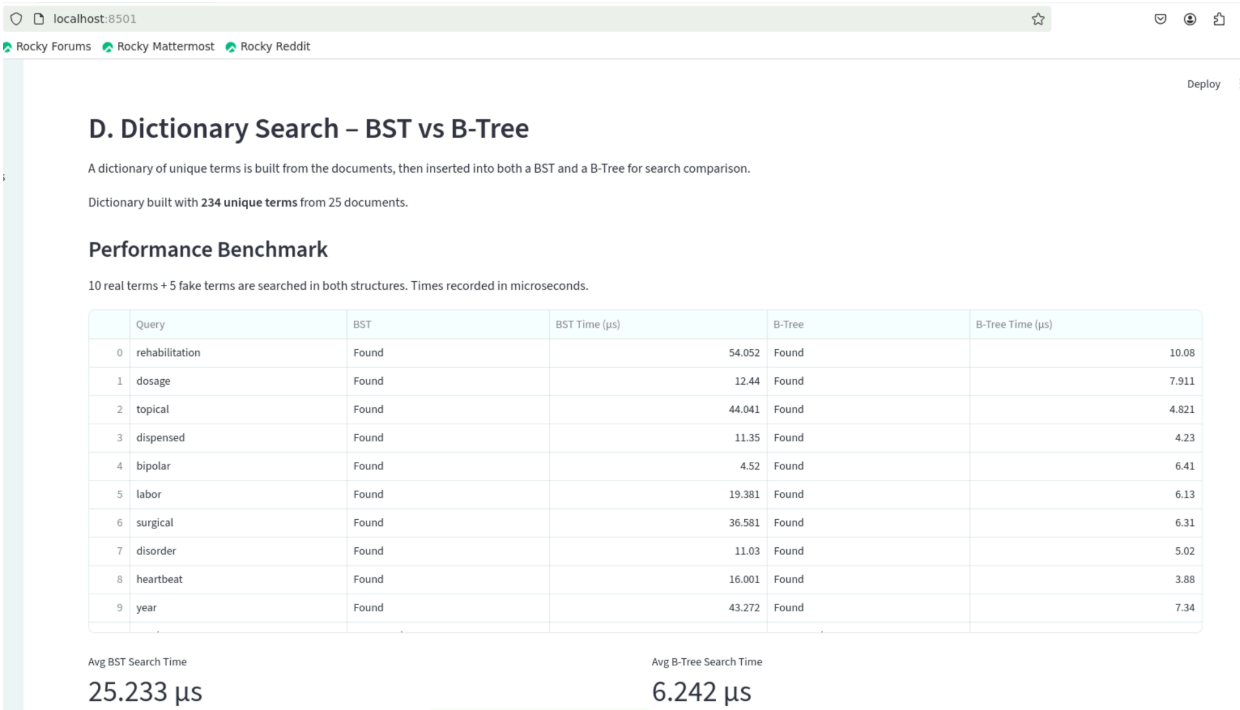
Performance Analysis

Results show:

- Similar performance for small collections.
- B-Tree remains stable as collection size increases.
- BST performance deteriorates with skewed insertion order.

Conclusion:

B-Tree is preferred for production-scale retrieval systems.



D. Dictionary Search – BST vs B-Tree

A dictionary of unique terms is built from the documents, then inserted into both a BST and a B-Tree for search comparison.

Dictionary built with **234 unique terms** from 25 documents.

Performance Benchmark

10 real terms + 5 fake terms are searched in both structures. Times recorded in microseconds.

	Query	BST	BST Time (µs)	B-Tree	B-Tree Time (µs)
0	rehabilitation	Found	54.052	Found	10.08
1	dosage	Found	12.44	Found	7.911
2	topical	Found	44.041	Found	4.821
3	dispensed	Found	11.35	Found	4.23
4	bipolar	Found	4.52	Found	6.41
5	labor	Found	19.381	Found	6.13
6	surgical	Found	36.581	Found	6.31
7	disorder	Found	11.03	Found	5.02
8	heartbeat	Found	16.001	Found	3.88
9	year	Found	43.272	Found	7.34

Avg BST Search Time
25.233 µs

Avg B-Tree Search Time
6.242 µs

---

9. Tolerant Retrieval

Users often submit incomplete or misspelled queries.

The system incorporates multiple tolerant retrieval mechanisms.

9.1 Wildcard Search

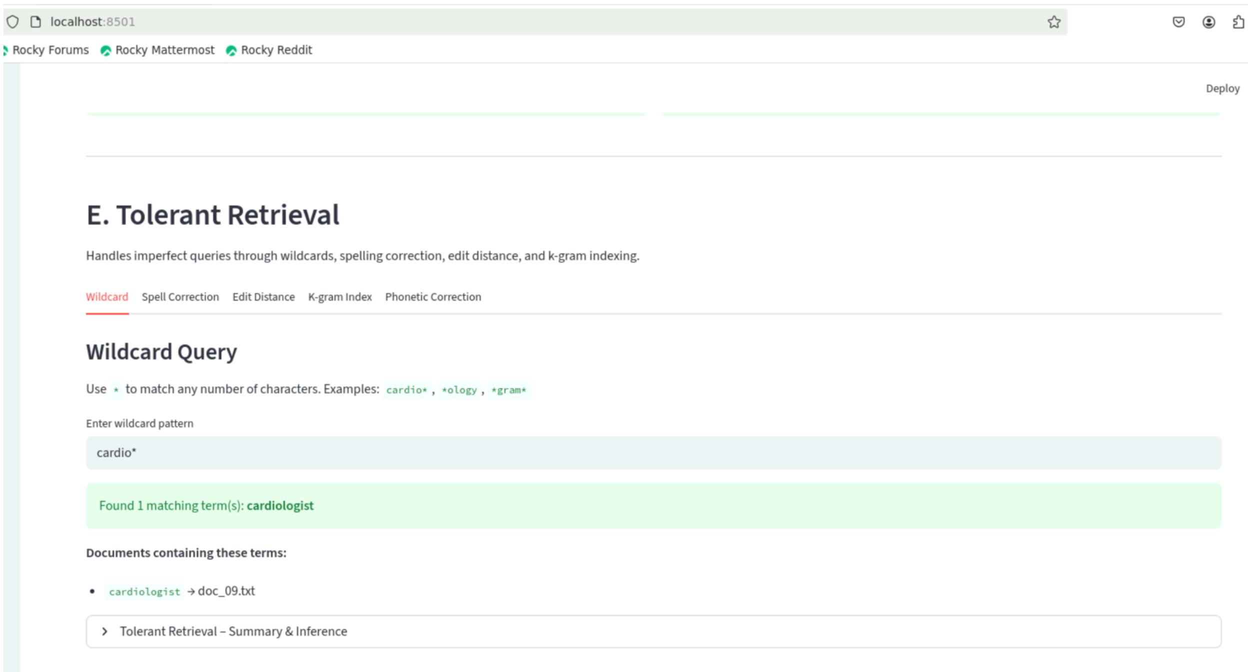
Supports pattern matching.

Examples:

cardio*
*ology
gram

Benefits:

- Flexible searching.
- Handles partial terms.



9.2 Edit Distance

Measures the minimum operations required to transform one word into another.

Operations:

- Insert
- Delete
- Substitute

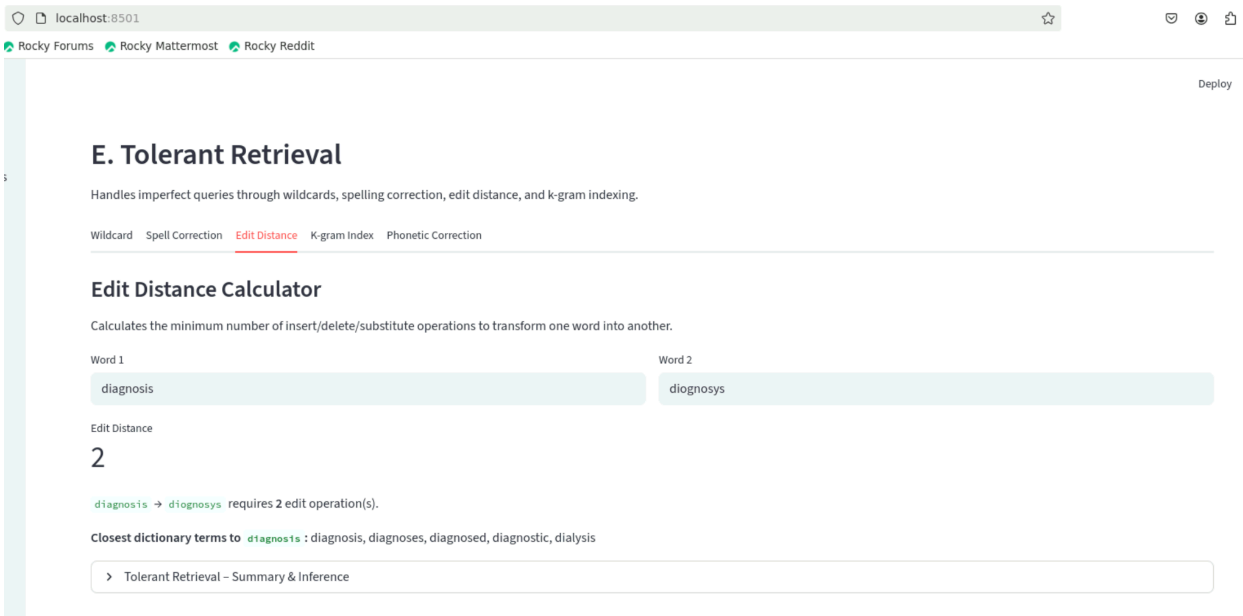
Example:

diagnosis
diognosis

Edit Distance = 1

Benefits:

- Detects spelling errors.



9.3 K-Gram Index

Breaks words into character sequences.

Example:

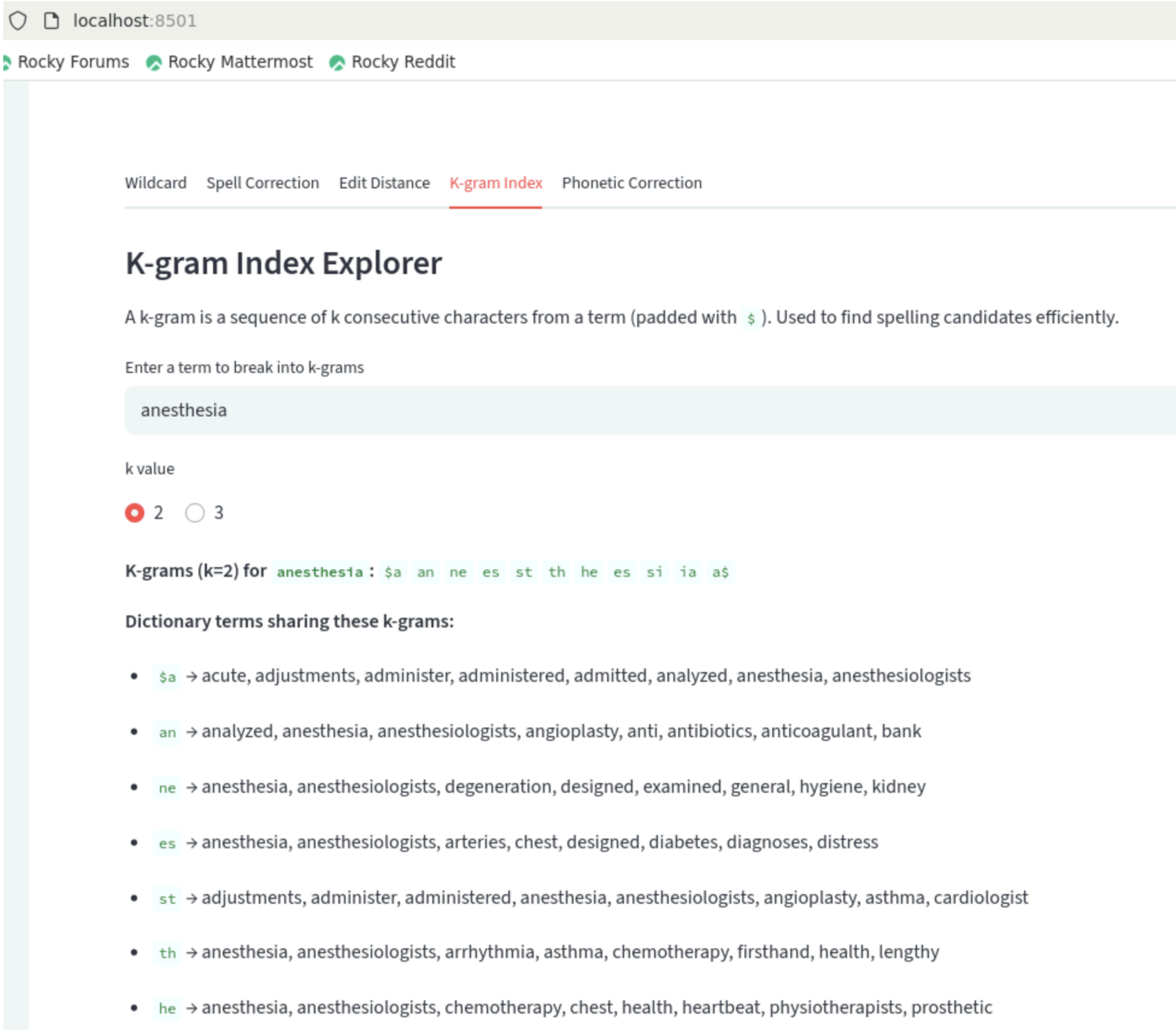
anesthesia

2-grams:

$a
an
ne
es
st
th
he
es
si
ia
a$

Benefits:

- Efficient candidate generation.
- Reduces search space.



9.4 Spell Correction

Combines:

1. K-Gram candidate generation
2. Edit Distance ranking

Example:

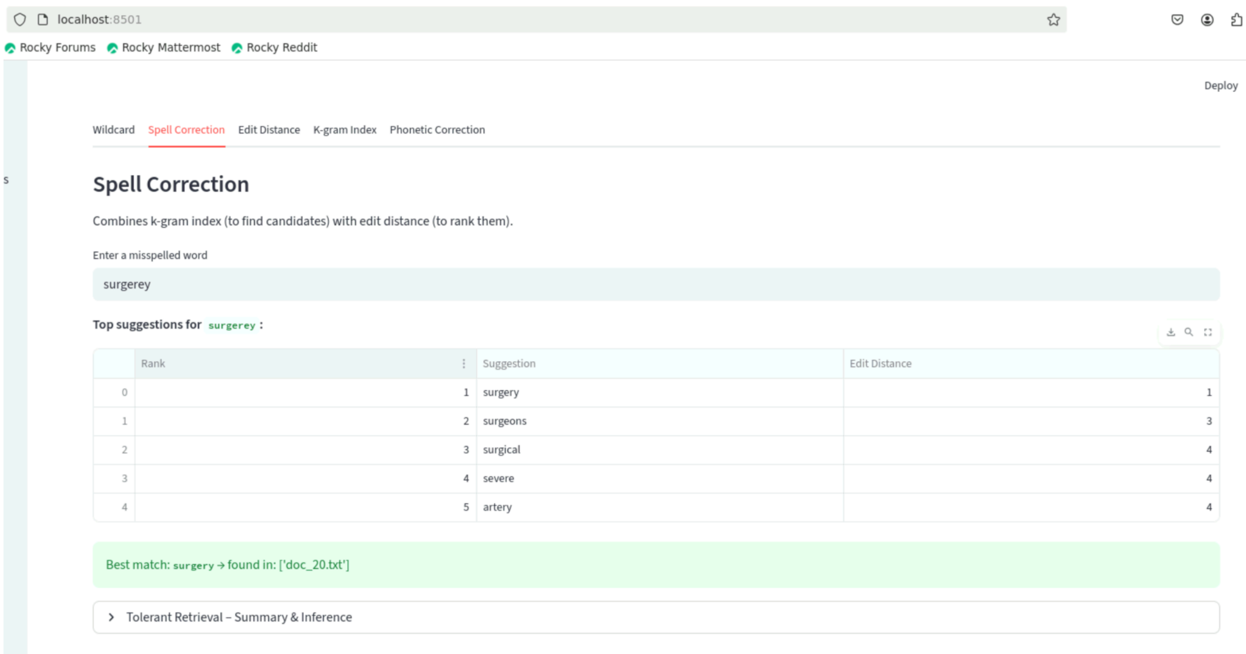
surgerey

Suggested Correction:

surgery

Benefits:

- High correction accuracy.
- Efficient retrieval.



9.5 Soundex Phonetic Matching

Groups words based on pronunciation.

Examples:

Smith
Smyth

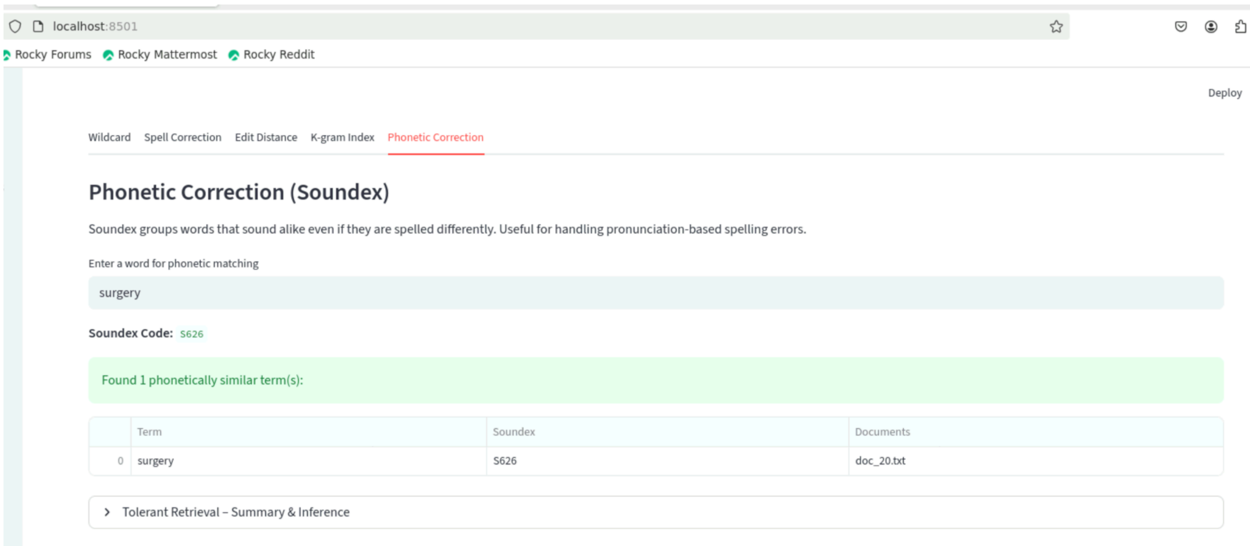
Both generate similar Soundex codes.

Benefits:

- Handles pronunciation-based spelling errors.

Limitations:

- Less effective for specialized medical terminology.



---

10. Experimental Results

Observations obtained from testing:

Preprocessing

Best configuration:

- Lowercasing
- Stopword Removal
- Lemmatization

This combination produced the most meaningful retrieval results.

Phrase Queries

Positional Index consistently outperformed Biword Index for multi-word phrases.

Dictionary Structures

B-Tree demonstrated superior scalability and stability.

Tolerant Retrieval

The combination of:

- K-Gram Index
- Edit Distance

provided the most accurate spelling corrections.

---

11. Conclusion

This project successfully demonstrates the implementation of core Information Retrieval concepts through an interactive Streamlit application. The system supports document preprocessing, indexing, phrase searching, dictionary lookup, and tolerant retrieval mechanisms. Experimental analysis shows that lemmatization improves retrieval quality, positional indexing provides accurate phrase matching, and B-Trees offer better scalability than BSTs. The tolerant retrieval module effectively handles misspellings and partial queries through K-grams, edit distance, wildcard search, and phonetic matching.

The developed system provides a practical demonstration of classical Information Retrieval techniques and serves as a strong foundation for future extensions involving ranked and semantic search.